SIGK - Projekt 2
Low light image enhancement

Autor: Łukasz Dąbała

1 Wymagania projektu

W ramach projektu należy stworzyć program, który będzie realizował opisane w temacie funkcje. Projekt jest zadaniem zespołowym, gdzie każdy zespół składa się z 2 osób.

Głównym językiem programowania powinien być język Python wraz z frameworkiem przeznaczonym do sieci neuronowych: Pytorch.

Za projekt można uzyskać maksymalnie $x \times 12p.$, gdzie x to liczba osób w zespole. Każdy z członków zespołu może dostać maksymalnie 12 punktów.

Ocenie w ramach projektu podlegają:

1. Działanie programu - realizacja funkcji (8 p.)
2. Dokumentacja dokonanych eksperymentów oraz wizualizacja wyników (4 p.)

Projekt uznaje się za oddany w momencie prezentacji go prowadzącemu.

2 Ćwiczenie

Metoda Celem projektu jest stworzenie sieci neuronowej, która będzie w stanie zrekonstruować zdjęcie zrobione przy ekstremalnie małym oświetleniu oraz szumie tzn. w ramach projektu należy spróbować rozwiązać problem: low light image enhancement. Aby rozwiązać problem należy zaproponować architekturę, wytrenować ją oraz dokonać ewaluacji stworzonego modelu. Żeby było możliwe wykonanie tego polecenia, należy zrozumieć fizykę powstawania obrazu cyfrowego na matrycy aparatu.

Dataset Jako zbiór danych należy wykorzystać zbiór RAISE: https://loki.disi.unitn.it/RAISE/download.html. Cały zbiór jest ogromny, dlatego wybieramy tylko podzbiór, który obejmuje obrazy wnętrz (*Indoor photos*) zrobione modelem aparatu: **Nikon D7000**. Ze zbioru wykorzystujemy tylko 100 pierwszych obrazów. Dokonujemy ewaluacji na 20 ostatnich z nich.

Plik CSV ze spisem zdjęć znajduje się również w archiwum dołączonym do instrukcji.

W ramach ćwiczenia należy też odpowiednio przygotować obrazy do treningu sieci. W przestrzeni liniowej światła należy obniżyć ekspozycję o -3 EV (8 razy mniej światła), a następnie należy dodać fizyczny model szumu aparatu (szum fotonowy Poissona oraz elektroniczy Gaussa). Dopiero taki obraz powinien być przekształcony z wykorzystaniem krzywej gamma.

Metryki W celu weryfikacji wyników należy wykorzystać PSNR, SSIM oraz LPIPS dla wygenerowanych obrazów.

Fragment kodu Ze względu na konieczność operacji na konieczność pobrania dużej liczby plików poniżej fragment kodu, który powinien to odrobinę uprościć:

```
import os
import csv
import requests

def download_raise_nef(csv_path, output_dir, limit=None):
    """
    Downloads NEF files from the RAISE dataset based on a CSV file.
    The 'limit' parameter specifies the maximum number of files to process.
    """
    os.makedirs(output_dir, exist_ok=True)

    with open(csv_path, mode='r', encoding='utf-8') as file:
        csv_reader = csv.DictReader(file)

        for i, row in enumerate(csv_reader):
            if limit is not None and i >= limit:
                break
```

```
            nef_url = row['NEF']
            filename = row['File']
            if not filename.upper().endswith('.NEF'):
                filename += '.NEF'

            filepath = os.path.join(output_dir, filename)

            if os.path.exists(filepath):
                print(f"Skipping {filename} - already exists.")
                continue

            print(f"Downloading {filename}...")
            response = requests.get(nef_url, stream=True)

            with open(filepath, 'wb') as out_file:
                for chunk in response.iter_content(chunk_size=8192):
                    out_file.write(chunk)
```

Ze względu również na konieczność generacji zbioru danych, można wykorzystać też skrypt poniżej - konieczne jest uzupełnienie przekształcania obrazu w 3 metodach.

```
import os
import glob
import rawpy
import imageio
import numpy as np

def get_ground_truth(raw):
    """
    Generates the properly exposed, full-color sRGB image (Ground Truth).
    half_size=True is used to speed up processing during training/experiments.
    """
    return raw.postprocess(use_camera_wb=True, half_size=True)

def add_physical_noise(linear_img, alpha=0.001, sigma=0.005):
    """
    Simulates physical camera sensor noise.
    alpha: controls Poisson (photon) noise
    sigma: controls Gaussian (electronic read) noise
    """
    noisy_img = np.random.poisson(linear_img / alpha) * alpha
    noisy_img += np.random.normal(0, sigma, linear_img.shape)
    return np.clip(noisy_img, 0.0, 1.0)

def get_linear_image(raw):
```

```
    """
    Extracts the linear image from RAW data.
    Disables auto-brightness and gamma correction.
    Returns a float32 image in the range [0.0, 1.0].
    """
    linear_16bit = raw.postprocess(
        use_camera_wb=True,
        gamma=(1, 1),
        no_auto_bright=True,
        output_bps=16,
        half_size=True
    )
    return linear_16bit.astype(np.float32) / 65535.0

def reduce_exposure(linear_img, ev_shift):
    """
    TODO: Simulate exposure reduction.
    Hint: A change of -1 EV halves the amount of light.
    Calculate the scale factor based on 'ev_shift' and multiply the image by it.
    """
    raise NotImplementedError("Implement exposure reduction logic")

def apply_gamma_correction(img, gamma=2.2):
    """
    TODO: Apply sRGB gamma correction.
    Hint: The simplified formula for gamma correction is image ^ (1 / gamma).
    """
    raise NotImplementedError("Implement gamma correction logic")

def convert_to_8bit(img):
    """
    TODO: Convert the float image (range 0.0 to 1.0) to an 8-bit integer (range 0 to 255).
    Ensure the returned array has the dtype np.uint8.
    """
    raise NotImplementedError("Implement 8-bit conversion logic")

def get_low_light(raw, ev_shift=-3.0):
    """
    Generates the degraded, low-light input image.
    Students must complete the helper functions above to make this pipeline work.
    """
    # Step 1: Get linear image data
    linear_img = get_linear_image(raw)

    # Step 2: Mathematically reduce the exposure
    dark_linear = reduce_exposure(linear_img, ev_shift)
```

```
    # Step 3: Add physical sensor noise
    noisy_dark_linear = add_physical_noise(dark_linear)

    # Step 4: Apply gamma curve for correct display
    dark_srgb = apply_gamma_correction(noisy_dark_linear)

    # Step 5: Convert back to standard 8-bit format
    final_8bit = convert_to_8bit(dark_srgb)

    return final_8bit

def process_single_file(file_path, output_gt_dir, output_dark_dir, ev_shift):
    base_name = os.path.splitext(os.path.basename(file_path))[0]
    gt_path = os.path.join(output_gt_dir, f"{base_name}_gt.png")
    dark_path = os.path.join(output_dark_dir, f"{base_name}_input.png")

    if os.path.exists(gt_path) and os.path.exists(dark_path):
        print(f"Skipping {base_name} - already processed.")
        return

    print(f"Processing {base_name}...")
    try:
        with rawpy.imread(file_path) as raw:
            img_gt = get_ground_truth(raw)
            img_dark = get_low_light(raw, ev_shift)

        imageio.imsave(gt_path, img_gt)
        imageio.imsave(dark_path, img_dark)
    except Exception as e:
        print(f"Error processing {base_name}: {e}")
```

3 Przedstawienie wyników

Oprócz przedstawienia wyników wizualnych tzn. stworzonych rekonstrukcji, należy również dokonać porównania z wykorzystaniem metryk PSNR, SSIM oraz LPIPS z obrazem ground truth (tzn. z tym który jest wygenerowany w ramach skryptu). Żeby sprawdzić, czy rzeczywiście sieć jest lepsza niż proste rozwiązanie należy również stworzyć rozwiązanie baseline tzn. w tym przypadku należy zastosować wyrównanie histogramu na obrazie.

W tabeli powinny znaleźć się 2 wiersze z wynikami:

Metoda	**PSNR**	**SSIM**	**LPIPS**
baseline	<wartość>	<wartość>	<wartość>
network	<wartość>	<wartość>	<wartość>

Tabela 1: Przykład tabeli do przedstawienia wyników

Należy przeanalizować osiągnięte wyniki i wygenerowane obrazy oraz zastanowić się nad specjalnymi przypadkami (kolory, przepalenia).